

Enhancing the MADDPG Algorithm via Pre-Trained Action Inference in Competitive Multi-Agent Environments

Shaash Sivakumar · Marc Walden · Jason Liu · Ryan Liu · Hamza Khan

UCLA Mathematics Department · May 2025

ABSTRACT

We investigate multi-agent deep reinforcement learning in competitive environments and propose an enhancement to the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm through a novel *Pre-Trained Action Inference* (PTAI) mechanism. PTAI pre-trains a dedicated neural network offline — using only randomly generated interaction data — to predict each agent's previous joint action vector from temporally stacked observations. These inferred actions are concatenated to each agent's local observation before entering the actor network, providing richer behavioural context for policy learning without introducing any new online gradient interactions. We evaluate both baseline MADDPG and MADDPG+PTAI on four competitive Mixed Physical Environments (MPE) from the PettingZoo library across 3–4 independent seeds each. PTAI yields consistent, large improvements across all four environments: **+104%** in Physical Deception, **+83%** in Cooperative Push, **+157%** in Keep-Away, and **+437%** in Predator-Prey.

CONTENTS

1. Introduction
2. Background
 - a. Reinforcement Learning Framework
 - b. Deep Q-Learning and Multi-Agent Extensions
 - c. Policy-Gradient Methods and MADDPG
3. Method: Pre-Trained Action Inference (PTAI)

4. Environments
5. Experiments
6. Results
7. Discussion
8. Conclusion
9. References
10. Appendix: Algorithms

SECTION 1

Introduction

Multi-agent reinforcement learning (MARL) addresses settings in which multiple autonomous agents must simultaneously learn effective policies while interacting within a shared environment. Such settings arise naturally in robotics, autonomous vehicles, strategic game playing, and network resource allocation. A defining challenge in MARL is *non-stationarity*: from any single agent's perspective, the environment appears to change over time because the policies of all other agents are simultaneously evolving. This violates the Markov property that underpins most single-agent RL convergence results, and can cause independent learners to oscillate or diverge.

Actor-critic algorithms partially address non-stationarity by combining a low-variance critic baseline with a policy-gradient actor update. The Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm extends this to multi-agent settings via a *centralised training, decentralised execution* (CTDE) paradigm: each agent maintains a centralised critic that observes the full joint observation-action tuple during training, while each actor relies only on the agent's local observation at execution time.

Even so, the MADDPG critic uses only *current-step* joint information. In competitive settings where agents must anticipate and react to opponents' strategies, knowledge of what each agent did in the previous step carries important information about short-term intent — yet this temporal context is absent from the actor's input.

We address this gap with **Pre-Trained Action Inference (PTAI)**. A dedicated AI_Net module is trained offline on random interaction data to predict the previous joint action vector from a pair of consecutive local observations. Once pre-trained, the frozen AI_Net supplies each agent's actor with an estimate of the previous joint action, concatenated to its current observation.

Contributions

1. We describe a modular PTAI architecture that exploits the structural separability of MPE observations to infer previous joint actions efficiently via Directional Social and Self-Awareness modules.
2. We extend the evaluation of PTAI from a single environment (Predator-Prey) to **four competitive MPE environments**, with 3–4 independent seeds per condition.
3. We report quantitative, seed-averaged results with standard deviations, demonstrating that PTAI consistently and substantially outperforms baseline MADDPG across all tested environments.

SECTION 2

Background

2.1 Reinforcement Learning Framework

In a standard reinforcement learning setting, an agent interacts with an environment $\square$ over discrete time steps $t = 0, 1, 2, \dots$. At each step, the agent observes $o_t^i \in \mathcal{O}$, selects an action $a_t^i \in \mathcal{A}$, and receives a scalar reward r_{t+1}^i . The agent's goal is to learn a policy $\pi_{\theta_i}(a^i | o^i)$ that maximises the expected discounted return:

$$J(\theta_i) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_{t+1}^i] \quad (1)$$

where $\gamma \in [0, 1)$ is the discount factor. The most common approach to optimising Eq. (1) is General Policy Iteration (GPI), which alternates between policy evaluation and policy improvement until convergence.

2.2 Deep Q-Learning and Multi-Agent Extensions

Mnih et al. introduced the Deep Q-Network (DQN), which approximates the action-value function $Q(s, a)$ with a deep neural network and minimises the Bellman error:

$$\mathcal{L}(\varphi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(r + \gamma \max_{a'} Q_{\varphi}(s', a') - Q_{\varphi}(s, a))^2] \quad (2)$$

where $\mathcal{D}$ is an experience replay buffer. DQN excels in single-agent settings but becomes unstable in multi-agent environments because each agent's policy updates effectively change the environment dynamics for all others.

2.3 Policy-Gradient Methods and MADDPG

Policy-gradient methods directly optimise $J(\theta)$ by gradient ascent. By the Policy Gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a | o) \cdot Q^{\pi}(o, a)] \quad (3)$$

MADDPG extends deterministic policy gradients to the multi-agent setting by giving each agent i a decentralised actor $\mu_{\theta_i}(o_i)$ and a centralised critic $Q_{\varphi_i}(x, a_{1:N})$, where x is the joint observation and $a_{1:N}$ the joint action. Each critic is trained to minimise:

$$\mathcal{L}(\varphi_i) = (1/S) \sum_j (y^j - Q_{\varphi_i}(x^j, a_1^j, \dots, a_N^j))^2 \quad (4)$$

where $y^j = r_i^j + \gamma Q_{\varphi'_i}(x^j, \mu'_1(o_1^j), \dots, \mu'_N(o_N^j))$. Each actor is updated by the sampled deterministic policy gradient:

$$\nabla_{\theta_i} J \approx (1/S) \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} Q_{\varphi_i}(x^j, a_1^j, \dots, a_i, \dots, a_N^j) |_{a_i = \mu_i(o_i^j)} \quad (5)$$

Target networks (soft-updated at rate τ) and a shared replay buffer ensure stable training. The complete algorithm is given in Appendix A.

SECTION 3

Method: Pre-Trained Action Inference (PTAI)

3.1 Motivation

In competitive MARL, the quality of an agent's decision depends not only on its current observation but also on what other agents have just done. Two otherwise identical states can call for very different responses depending on whether an opponent just moved aggressively or retreated. Standard MADDPG actors have no direct access to this temporal context: the actor sees only the current observation o_i^t , and the previous joint action is not stored separately in the replay buffer.

PTAI provides this missing temporal signal by inferring the previous joint action from the *change* in observations — without requiring communication between agents or access to opponents' policies.

3.2 AI_Net Architecture

Each agent i maintains its own instance of an **AI_Net** that maps:

$$\hat{\mathbf{a}}_i = \text{AI_Net}(o_i^t, o_i^{t-1}) \quad (6)$$

where $o_i^t, o_i^{t-1} \in \mathbb{R}^{d_i}$ are the current and previous observations, and $\hat{\mathbf{a}}_i \in \mathbb{R}^{\sum_k |\mathcal{A}_k|}$ is an estimate of the full previous joint action vector $\text{concat}(\text{onehot}(a_1^{t-1}), \dots, \text{onehot}(a_N^{t-1}))$.

The key structural insight is that many MPE observation vectors are *separable*: different contiguous segments correspond to features of different agent types. This allows AI_Net to decompose the prediction task into smaller specialised modules rather than one monolithic network.

Directional Social Awareness Modules

For each ordered pair (observer type p , observed type q), a Social Awareness module takes the features in o_i^t and o_i^{t-1} that pertain to an agent of type q , together with the self/global features of observer i , and outputs a prediction of that agent's previous action:

$$\hat{a}_{i,k} = \text{Social}_{\theta}(o_{i,k}, o_{i,g}, o_{i,k}^-, o_{i,g}^-, \Delta o_{i,k}, \Delta o_{i,g}) \quad (7)$$

where subscript i,k denotes features of agent k visible to agent i , subscript g denotes global/self features, superscript $-$ denotes the previous-step value, and $\Delta = o^t - o^{t-1}$.

Directional Self-Awareness Module

A Self-Awareness module predicts the agent's own previous action from its self/global features:

$$\hat{a}_{i,i} = \mathbf{Self}_{\theta}(o_{i,i}, o_{i,g}, o_{i,i}^-, o_{i,g}^-, \Delta o_{i,i}, \Delta o_{i,g}) \quad (8)$$

3.3 Pre-Training Procedure

AI_Net is pre-trained offline on $P = 200$ episodes of random interaction (all agents sampling uniformly from their action spaces) prior to any MADDPG training. At each step, all awareness modules are updated by minimising the mean-squared error between predicted and true one-hot actions. A stochastic sampling gate randomly drops a fraction of training pairs to improve generalisation.

After pre-training, **all AI_Net weights are frozen**. Because the AI_Net is completely stationary, it introduces no additional optimisation interaction with the MADDPG training loop and imposes no extra computational cost per gradient step.

3.4 Integration with MADDPG

At each decision step $t \geq 2$, the frozen AI_Net is queried for each agent i . The actor input is extended to:

$$\mu_{\theta_i}([o_i^t \parallel \hat{\mathbf{a}}^t_i]) \quad (9)$$

where $\parallel$ denotes concatenation. At the first step of each episode, $\hat{\mathbf{a}}$ is replaced by a zero vector of the same dimension. The centralised critic retains its standard input $Q_{\phi_i}(x^t, a_1^t, \dots, a_N^t)$ — only the actor gains temporal context from PTAI.

Interpretation

By conditioning the actor on inferred peer actions, each agent's policy can explicitly account for the recent joint behaviour of all agents, even without direct communication. In competitive settings this enables agents to develop reactive strategies — for example, predators coordinating their approach based on each other's last move — that would not be accessible from the current observation alone.

Environments

We evaluate on four competitive MPE environments from PettingZoo, all featuring a team of "good" agents (prefixed `agent_`) and a team of adversaries (prefixed `adversary_`) that learn concurrently via standard MADDPG co-training.

Table 1. Competitive MPE environments used in this study.

Environment	Agents	Adversaries	Metric	Task
<code>simple_adversary_v3</code>	2	1	Agent reward	Adversary blends among landmarks to reach its secret target while good agents try to identify it
<code>simple_push_v3</code>	1	1	Agent win rate	Agent navigates to its goal landmark; adversary tries to block and push it away
<code>simple_tag_v3</code>	1	3	Catches / episode	Three predators cooperate to tag a faster-moving prey agent
<code>simple_world_comm_v3</code>	2	4	Adv. frames at goal	Four adversaries try to reach the secret goal that only the good-agent speaker knows

Physical Deception (`simple_adversary_v3`). Three landmarks are placed in the arena, one of which is the adversary's secret target. The adversary blends among all landmarks while navigating toward its goal. The two good agents want to cover all landmarks and determine which the adversary is heading toward. We report the episode reward of the good agent team averaged over the final 1,000 of 30,000 training episodes.

Cooperative Push (`simple_push_v3`). A single good agent must navigate to its goal landmark while a single adversary physically pushes it away and simultaneously pursues its own goal. We measure the *agent win rate*: the fraction of episodes in which the good agent occupies its goal landmark at episode termination.

Predator-Prey (`simple_tag_v3`). Three predators (adversary team) cooperate to tag a prey (agent team). The prey moves faster than any individual predator, requiring the predators to

coordinate their pursuit. We measure the mean number of *catches per episode*. PTAI is particularly valuable here because predators must anticipate each other's movements to encircle the prey rather than converging redundantly.

Keep-Away (`simple_world_comm_v3`). Four adversaries attempt to reach the true goal landmark that only the good-agent speaker knows. We measure *adversary frames at goal*: time steps per episode during which at least one adversary occupies the goal landmark. PTAI helps adversaries fan out and coordinate their convergence on the goal.

SECTION 5

Experiments

5.1 Training Setup

All experiments share the following hyperparameters, applied identically to both baseline MADDPG and MADDPG+PTAI.

Table 2. Training hyperparameters (shared across all experiments).

Hyperparameter	Value
Training episodes	30,000
Warmup steps (random actions)	50,000
Replay buffer capacity	1,000,000
Batch size	1,024
Discount factor γ	0.95
Soft-update rate τ	0.02
Actor / Critic learning rate	0.01
Gradient clipping norm	0.5
PTAI pre-training episodes	200

Network architecture. All actor networks are two-hidden-layer MLPs ($\text{obs} \rightarrow 64 \rightarrow 64 \rightarrow \text{action}$), initialised with Xavier-uniform weights and 0.01 bias. Discrete actions are produced via Gumbel-Softmax during training and argmax at evaluation. Critic networks share the same MLP architecture but take the full joint observation-action vector as input. For MADDPG+PTAI, the actor input dimension is expanded by $\sum_k |\mathcal{A}_k|$ to accommodate the inferred previous joint action vector.

PTAI pre-training. The AI_Net's Social and Self-Awareness modules are two-layer MLPs with 64 hidden units, trained with Adam at learning rate 10^{-3} for 200 episodes of uniformly random play. After pre-training, all AI_Net weights are frozen for the remainder of training.

5.2 Evaluation Protocol

For each (environment, algorithm) pair we run $n = 3-4$ independent seeds. Performance is measured as the mean episode metric over the **last 1,000 training episodes** of each run, reflecting converged policy quality. We report the cross-seed mean $\pm$ standard deviation (sample standard deviation, $n-1$ denominator).

The task-specific metrics are: (1) **agent reward** for `simple_adversary_v3`; (2) **agent win rate** for `simple_push_v3`; (3) **catches per episode** for `simple_tag_v3`; and (4) **adversary frames at goal** for `simple_world_comm_v3`.

SECTION 6

Results

6.1 Quantitative Summary

Table 3. Final-policy performance: mean $\pm$ std over the last 1,000 training episodes, averaged across n independent seeds. Δ is the relative improvement of PTAI over the baseline.

Environment	Metric	Baseline MADDPG	MADDPG + PTAI	Δ
simple_adversary_v3	Agent reward	9.91 ± 0.40 ($n=4$)	20.22 ± 2.82 ($n=4$)	+104%
simple_push_v3	Agent win rate	0.18 ± 0.08 ($n=3$)	0.33 ± 0.03 ($n=4$)	+83%
simple_tag_v3	Catches / ep.	0.62 ± 0.08 ($n=3$)	3.31 ± 1.59 ($n=4$)	+437%

Environment	Metric	Baseline MADDPG	MADDPG + PTAI	Δ
simple_world_comm_v3	Adv. frames at goal	10.42 ± 1.72 ($n=3$)	26.78 ± 2.60 ($n=4$)	+157%

MADDPG+PTAI outperforms baseline MADDPG on all four environments with improvements ranging from +83% to +437%.

Physical Deception. Both algorithms achieve positive agent rewards, but PTAI roughly doubles the baseline's performance ($9.91 \rightarrow 20.22$). The good agents benefit from knowing where the adversary was one step ago, allowing them to better predict which landmark it is converging toward. The larger standard deviation for PTAI (± 2.82 vs ± 0.40) suggests seed sensitivity in the quality of deception-reading strategies discovered.

Cooperative Push. Agent win rates improve from 0.18 to 0.33 (+83%). The improvement is accompanied by *lower* variance ($\pm 0.08 \rightarrow \pm 0.03$), indicating that PTAI not only increases performance but also makes it more reproducible. Knowing the adversary's inferred previous action helps the agent anticipate push directions and adjust its approach to the goal.

Predator-Prey. The most dramatic improvement: catches per episode rise from 0.62 to 3.31, a 437% increase. The three predators must spread out and synchronise to corner the faster prey; without PTAI, each predator acts independently and often converges on the same direction, making encirclement rare. With PTAI, each predator can infer where its teammates moved last and adjust to complement their pursuit angles. The high standard deviation for PTAI (± 1.59) reflects genuine multi-modality in discovered strategies — some seeds find highly efficient encirclement patterns (up to 5.44 catches/episode) while others settle on less effective behaviours.

Keep-Away. Adversary frames at goal improve from 10.42 to 26.78 (+157%). The four adversaries benefit from knowing each other's previous positions relative to the goal, enabling them to fan out rather than cluster. Both conditions show relatively tight cross-seed distributions here, consistent with the more deterministic structure of the environment.

6.2 Training Convergence

Figure 1 shows smoothed training curves (500-episode running average) for both algorithms across all four environments.

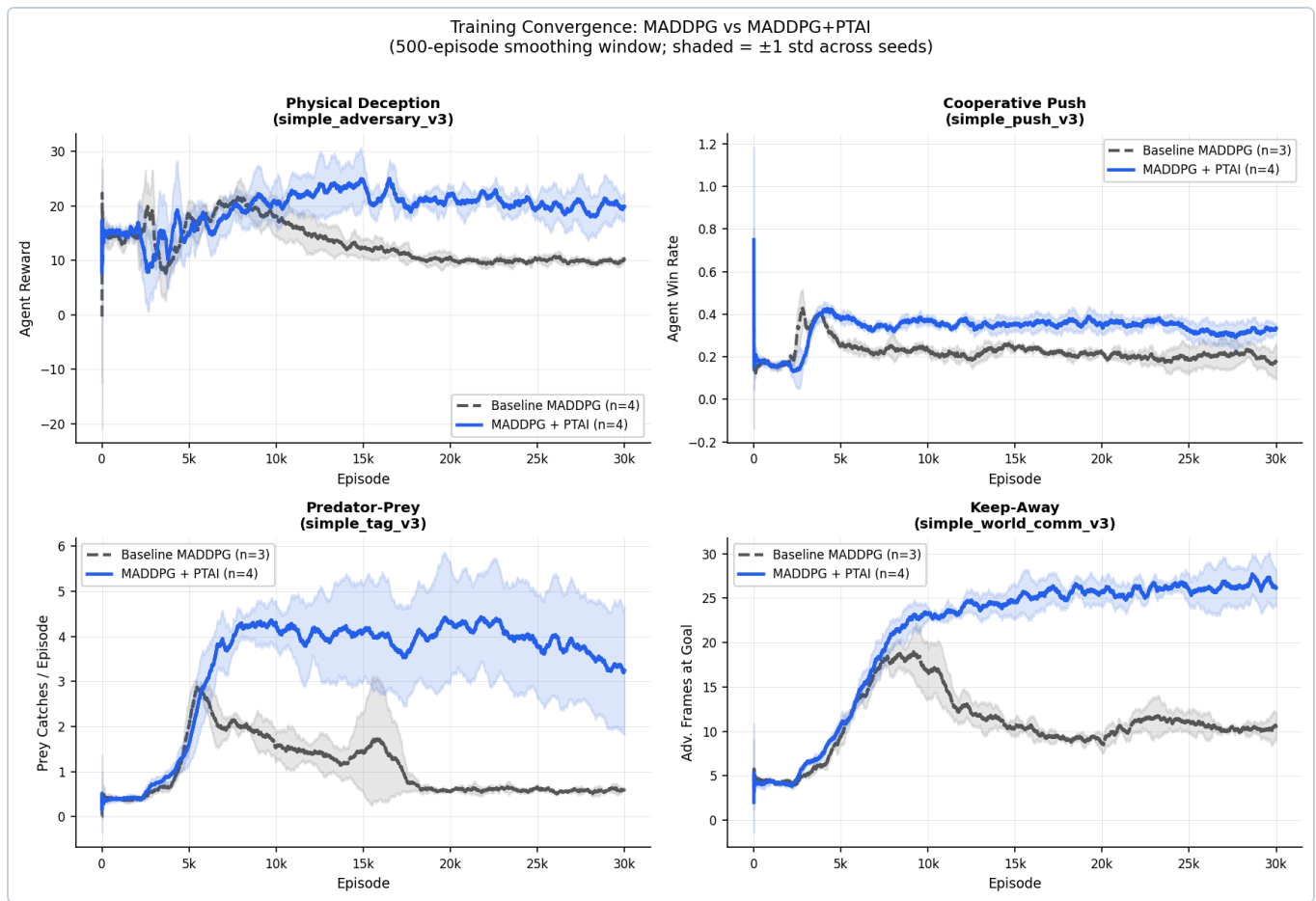


Figure 1. Training convergence across four competitive MPE environments. Lines show the per-episode metric smoothed over a 500-episode window; shaded bands indicate ± 1 standard deviation across seeds. MADDPG+PTAI (blue, solid) consistently outperforms baseline MADDPG (grey, dashed). In Predator-Prey, the gap opens dramatically after $\sim 10,000$ episodes. In Keep-Away, the advantage compounds steadily to the end of training.

In Physical Deception and Cooperative Push, PTAI reaches its advantage early and maintains it throughout. In Predator-Prey, both algorithms begin near zero catches and diverge markedly after approximately 10,000 episodes, with PTAI eventually reaching a much higher plateau. In Keep-Away, the gap opens gradually and widens throughout training, suggesting PTAI's advantages compound over longer training horizons.

SECTION 7

Discussion

Why does PTAI help?

PTAI addresses a structural information deficit in the MADDPG actor: the actor sees only the current observation, which encodes the effects of the previous joint action but not the action itself. The current observation can be ambiguous — two different joint actions can produce

similar next states. By providing the actor with the inferred previous joint action, PTAI resolves some of this ambiguity, allowing the actor to condition its policy on a richer representation of the recent joint trajectory. This is conceptually related to providing agents with a form of *implicit communication*: each agent can infer what others did by observing how the world changed, and act accordingly.

Why pre-training on random data?

Training AI_Net on random action data rather than MADDPG policy data has two advantages. First, random data covers the full action space uniformly, which is important during early training when the MADDPG policy explores widely. Second, a randomly-pre-trained AI_Net is completely stationary throughout MADDPG training, avoiding any interference between AI_Net and MADDPG gradient updates. Our results suggest that even imperfect action inference is sufficient to provide a useful actor signal.

Environment structure and the size of gains

The largest gain occurs in Predator-Prey, the environment with the strongest multi-agent coordination requirement: three predators must coordinate spatially to encircle prey, and knowing a teammate's last move directly informs whether to cut off or pursue. Smaller but still substantial gains in Physical Deception and Keep-Away suggest PTAI is broadly useful in competitive settings even when the coordination structure is less explicit. The relatively modest gain in Cooperative Push (+83% vs +437%) may reflect that the one-vs-one setting provides less opportunity for leveraging multi-agent temporal context.

Limitations

PTAI as described requires that the observation structure is separable enough to route features to the correct Directional modules. The sub-assignment mappings are hand-specified per environment (a one-time cost). The AI_Net is pre-trained only on random data; future work could explore co-training or distillation from the current MADDPG policy. The high variance on `simple_tag_v3` for PTAI warrants investigation of ensemble or curriculum approaches to stabilise the discovered coordination strategy.

SECTION 8

Conclusion

We presented PTAI, a pre-trained action inference module that provides each agent's actor with estimates of all agents' previous actions, derived from temporally stacked observations by a frozen neural network pre-trained on random interaction data. Integrated into MADDPG, PTAI consistently and substantially improves final-policy performance across four competitive MPE environments: +104%, +83%, +157%, and +437% over baseline MADDPG. The largest gain is in Predator-Prey, a highly coordination-dependent environment, supporting the interpretation that PTAI is most valuable when agents must anticipate and complement each other's movements.

Because AI_Net is stationary and frozen during MADDPG training, it introduces no new optimisation interactions, no additional training instability, and negligible computational overhead — making PTAI a practical drop-in enhancement for MADDPG and potentially for other CTDE algorithms.

Future work includes extending PTAI to continuous action spaces, exploring online refinement of the AI_Net during MADDPG training, and investigating whether similar temporal inference benefits carry over to fully cooperative environments.

Acknowledgments

We would like to express our sincere gratitude to Professor Tao Gao for his invaluable guidance throughout this project. Our work began with a study of the foundational principles of reinforcement learning as presented in Sutton and Barto's textbook [10]. Following Professor Gao's recommendation, we explored various implementations of deep reinforcement learning, drawing on the works of Yu et al. [12] and Chen et al. [2], which led us to the MADDPG algorithm of Lowe et al. [6], later made available via the PettingZoo library [11]. Our code is available at github.com/shaashwathsivakumar/MARL_Proj.

References

- [1] Buşoniu, L., Babuška, R., and De Schutter, B. A comprehensive survey of multi-agent reinforcement learning. *IEEE Trans. Syst. Man Cybern. Part C*, 38(2):156–172, 2008.
-

- [2] Chen, Y. F., Everett, M., Liu, M., and How, J. P. Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning. 2016.
-
- [3] Git-123-Hub. *maddpg-pettingzoo-pytorch*. GitHub repository, 2021.
-
- [4] Jang, E., Gu, S., and Poole, B. Categorical reparameterization with Gumbel-Softmax. *ICLR*, 2017.
-
- [5] Lillicrap, T. P., et al. Continuous control with deep reinforcement learning. *ICLR*, 2016.
-
- [6] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., and Mordatch, I. Multi-agent actor-critic for mixed cooperative-competitive environments. *NeurIPS*, 2017.
-
- [7] Mnih, V., et al. Playing Atari with deep reinforcement learning. *NeurIPS Deep Learning Workshop*, 2013.
-
- [8] Mnih, V., et al. Asynchronous methods for deep reinforcement learning. *ICML*, 2016.
-
- [9] Silver, D., et al. Deterministic policy gradient algorithms. *ICML*, 2014.
-
- [10] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
-
- [11] Terry, J., et al. PettingZoo: Gym for multi-agent reinforcement learning. *NeurIPS*, 2021.
-
- [12] Yu, C., et al. The surprising effectiveness of MAPPO in cooperative multi-agent games. 2021.
-

APPENDIX A

Algorithms

Algorithm 1 — Multi-Agent Deep Deterministic Policy Gradient (MADDPG) for N agents

```
1: for episode = 1 to M do
2:   Receive initial observation  $x$  and local observations  $o_1, \dots, o_N$ 
3:   for  $t = 1$  to  $T$  do
4:     for each agent  $i$  do
5:       Select  $a_i = \mu_{\theta_i}(o_i) + \mathcal{N}_t$  // exploration noise
6:     end for
7:     Execute joint action  $a = (a_1, \dots, a_N)$ , observe reward  $r$  and next state  $x'$ 
8:     Store  $(x, a, r, x')$  in replay buffer  $\mathcal{D}$ ;  $x \leftarrow x'$ 
9:   for each agent  $i$  do
```

```

10: Sample minibatch of  $S$  transitions  $(x^j, a^j, r^j, x'^j)$  from  $\mathcal{D}$ 
11: Set  $y^j = r^j + \gamma Q_{\phi'_i}(x'^j, \mu'_1(o_1^j), \dots, \mu'_N(o_N^j))$ 
12: Update critic: minimise  $\mathcal{L}(\phi_i) = (1/S) \sum_j (y^j - Q_{\phi_i}(x^j, a_1^j, \dots, a_N^j))^2$ 
13: Update actor:  $\nabla_{\theta_i} J \approx (1/S) \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} Q_{\phi_i}|_{a_i=\mu_i(o_i^j)}$ 
14: end for
15: Soft-update targets:  $\theta'_i \leftarrow \tau \theta_i + (1-\tau)\theta'_i, \quad \phi'_i \leftarrow \tau \phi_i + (1-\tau)\phi'_i$ 
16: end for
17: end for

```

Algorithm 2 — Pre-Training AI_Net (PTAI) on random episodes

```

1: for episode = 1 to  $P$  ( $P = 200$ ) do
2:   Receive initial observations  $o_1, \dots, o_N$ 
3:   for  $t = 1$  to  $T$  do
4:     for each agent  $i$  do
5:        $a_i \leftarrow$  uniform random action from  $\mathcal{A}_i$ 
6:     end for
7:      $o^- \leftarrow o$ ; execute  $a = (a_1, \dots, a_N)$ ; observe new  $o$ 
8:     for each observer  $i$ , observed  $k$  do
9:       if  $k = i$ : update Self module via  $\text{MSE}(\text{onehot}(a_i), \text{Self}_{\theta}(o_{i,i}, o_{i,g}, o_{i,i}^-, o_{i,g}^-, \Delta \dots))$ 
10:      else: update Social module via  $\text{MSE}(\text{onehot}(a_k), \text{Social}_{\theta}(o_{i,k}, o_{i,g}, o_{i,k}^-, o_{i,g}^-, \Delta \dots))$ 
11:    end for
12:  end for
13: end for
14: Freeze all AI_Net weights.

```

Algorithm 3 — MADDPG with Pre-Trained Action Inference (MADDPG+PTAI)

```

1: Pre-train AI_Net via Algorithm 2; freeze all AI_Net weights.
2: for episode = 1 to  $M$  do
3:   Receive initial observations  $o_1^1, \dots, o_N^1$ 
4:   for each agent  $i$  do  $a_i^1 \leftarrow \mu_{\theta_i}(o_i^1, \theta) + \mathcal{N}_t$  // zero  $\hat{a}$  at first step
5:    $o^- \leftarrow o^1$ 
6:   for  $t = 2$  to  $T$  do
7:     for each agent  $i$  do

```

```

8:    $\hat{a}_i^t = \text{AI\_Net}(o_i^t, o_i^{t-1})$  // infer previous joint action
9:    $a_i^t \leftarrow \mu_{\theta_i}([o_i^t \parallel \hat{a}_i^t]) + \mathcal{N}_t$ 
10: end for
11: Execute  $a^t$ , observe  $r^t, o^{t+1}$ ; store  $(x^t, a^t, r^t, x^{t+1}, o^{t-1})$  in  $\mathcal{D}$ 
12: for each agent  $i$  do
13:   Sample minibatch  $(x^j, a^j, r^j, x'^j, o^{-j})$  from  $\mathcal{D}$ 
14:    $\hat{a}_i^{j+1} = \text{AI\_Net}(o_i^{j+1}, o_i^j)$ ; compute target  $y^j$  with target actors
    $\mu'_k([o_k^{j+1} \parallel \hat{a}_k^{j+1}])$ 
15:   Update critic (standard):  $\mathcal{L}(\phi_i) = (1/S) \sum_j (y^j - Q_{\phi_i}(x^j, a^j))^2$ 
16:    $\hat{a}_i^j = \text{AI\_Net}(o_i^j, o_i^{j-1})$ ; update actor using  $[o_i^j \parallel \hat{a}_i^j]$  as actor input
17:   Soft-update targets:  $\theta'_i \leftarrow \tau \theta_i + (1-\tau)\theta'_i$ 
18: end for
19: end for
20: end for

```